Lecture 10: Linear Regression in Python

PSTAT100: Data Science — Concepts and Analysis

John Inston
johninston@ucsb.edu
University of California, Santa Barbara

April 25, 2026

🚁 Overview

Aims of the lecture

- Load a real dataset and perform **exploratory analysis** in preparation for regression.
- Fit a simple linear regression model using `statsmodels`.
- Read and interpret **every section** of the regression summary output.
- Visualise the fitted line alongside **confidence and prediction bands**.
- Carry out **residual diagnostics** to check the Gauss-Markov assumptions.
- Understand why diagnostics matter through **Anscombe's quartet**.

📚 Required Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import statsmodels.formula.api as smf
import statsmodels.api as sm
```

💅 Figure Styles

```
sns.set_style('whitegrid')
sns.set_palette('Set2')
```

The Dataset

🐧 Palmer Penguins

Data Recap

- Recall the **Palmer Penguins** dataset (Gorman, Williams, and Fraser 2014) from previous lectures.
  - Clean and well-studied dataset recorded from three penguin species on islands near Palmer Station, Antarctica.
- It contains physical measurements including **flipper length** and **body mass**.

```
# Load the dataset and drop rows with missing values in our key columns
penguins = sns.load_dataset('penguins')
penguins.head()
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	Male
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	Female
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	Female
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	Female

Modelling Question

Our modelling question

> **Can we predict a penguin's body mass from its flipper length?**

Keeping things simple

- We will be considering SLR with:
  - Predictor: `flipper_length_mm`
  - Response: `body_mass_g`
- We remove all missing values from these columns.

```
# Remove missing values
penguins = penguins.dropna(
    subset=['flipper_length_mm', 'body_mass_g']
)
# Basic summary
print(penguins.info())
```

```
<class 'pandas.DataFrame'>
Index: 342 entries, 0 to 343
Data columns (total 7 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   species            342 non-null    str
 1   island             342 non-null    str
 2   bill_length_mm     342 non-null    float64
 3   bill_depth_mm      342 non-null    float64
 4   flipper_length_mm  342 non-null    float64
 5   body_mass_g        342 non-null    float64
 6   sex                333 non-null    str
dtypes: float64(4), str(3)
memory usage: 21.4 KB
None
```

EDA Recap - Summary Statistics

Summary Statistics

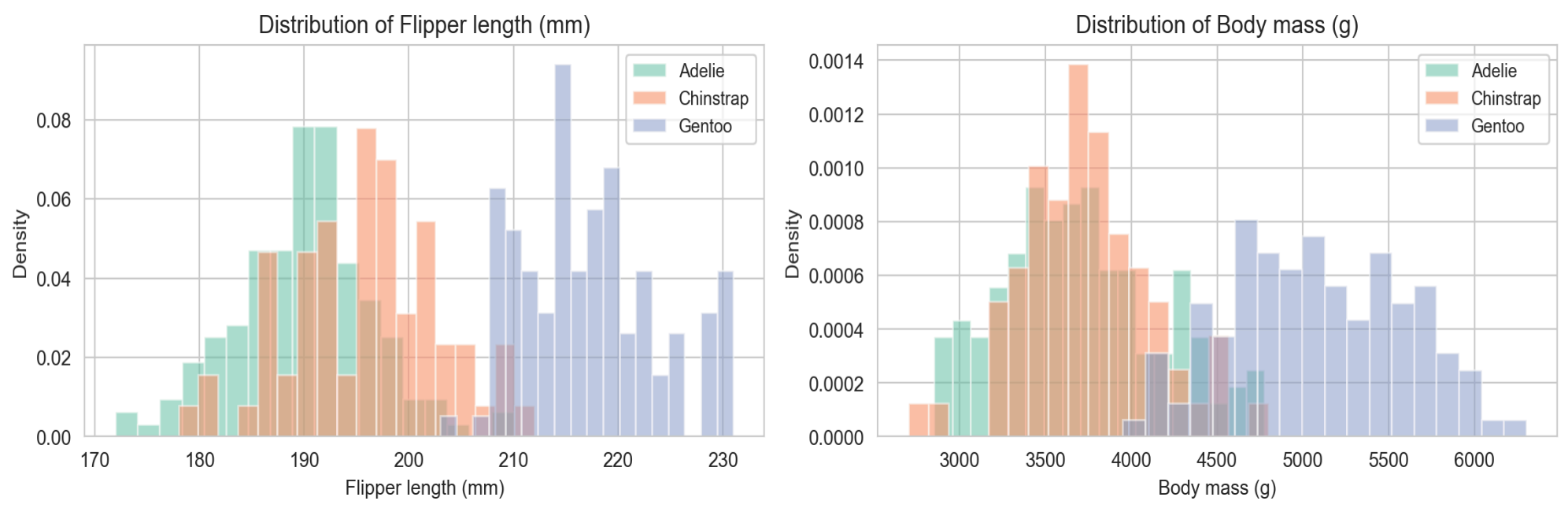
```
# Summary statistics for our two regression variables
penguins[['flipper_length_mm', 'body_mass_g']].describe().round(2)
```

	flipper_length_mm	body_mass_g
count	342.00	342.00
mean	200.92	4201.75
std	14.06	801.95
min	172.00	2700.00
25%	190.00	3550.00
50%	197.00	4050.00
75%	213.00	4750.00
max	231.00	6300.00

EDA Recap - Distributions

Histograms grouped by Species

```
fig, axes = plt.subplots(1, 2, figsize=(12, 3.5))
for ax, col, label in zip(axes,
  ['flipper_length_mm', 'body_mass_g'],
  ['Flipper length (mm)', 'Body mass (g)']):
    for sp, grp in penguins.groupby('species'):
        ax.hist(grp[col], bins=18, alpha=0.55, label=sp, density=True)
    ax.set_xlabel(label); ax.set_ylabel('Density')
    ax.set_title(f'Distribution of {label}')
    ax.legend(fontsize=9)
plt.tight_layout(); plt.show()
```



Distribution of each variable, coloured by species.

Exploratory Analysis for Regression

🔎 Scatterplot

The most important plot before regression

- Before performing regression we always produce a **scatter plot**.
    - We are looking for evidence of a linear relationship between the variables.

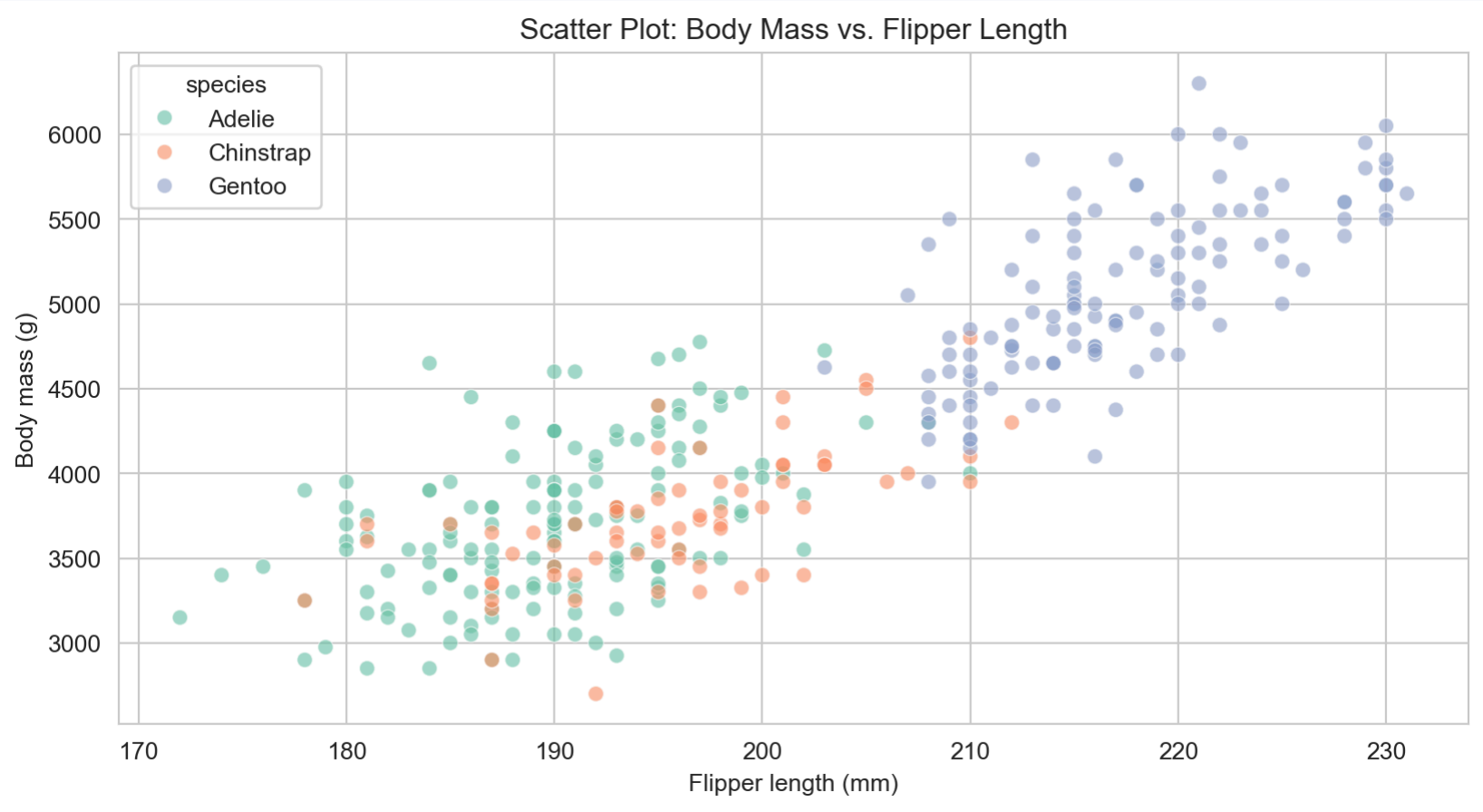
What to Look for in a Scatterplot

Before fitting any model, inspect the scatterplot for:

1. **Linearity** — does the relationship look approximately straight?
    - If strongly curved, a linear model may be inappropriate.
2. **Constant spread (homoscedasticity)** — does the vertical scatter stay roughly the same across x?
    - A fan-shape suggests heteroscedasticity (violation of GM3).
3. **Outliers and unusual observations** — any points far from the main cloud?
    - These can have a disproportionate influence on the fitted line.
4. **Clusters or subgroups** — distinct groups may suggest an omitted variable.
    - We can already see species clusters here — worth bearing in mind.

Example - Scatter Plot

```
sns.scatterplot(
  data=penguins, x='flipper_length_mm', y='body_mass_g',
  hue='species', alpha=0.6, s=40
)
plt.xlabel('Flipper length (mm)'); plt.ylabel('Body mass (g)')
plt.title('Scatter Plot: Body Mass vs. Flipper Length')
plt.show()
```



Body mass vs. flipper length. Points are coloured by species. A clear positive linear trend is visible.

Correlation

Quantifying the linear association

```python
# Pearson correlation between the two variables
r = penguins[
  ['flipper_length_mm', 'body_mass_g']
  ].corr()
print(r.round(4))
```

```
                   flipper_length_mm  body_mass_g
flipper_length_mm             1.0000       0.8712
body_mass_g                   0.8712       1.0000
```

- Correlation coefficient $r \approx 0.87$ indicates a strong positive linear association between flipper length and body mass.

```python
# Extract the scalar correlation coefficient
r_val = (
  penguins['flipper_length_mm']
  .corr(penguins['body_mass_g'])
)
print(f"r = {r_val:.4f}")
print(f"r² = {r_val**2:.4f}")
```

```
r = 0.8712
r² = 0.7590
```

- Flipper length explains about 76% of the variance in body mass ($R^2 \approx 0.76$) — a strong relationship, but not perfect.

Fitting SLR with statsmodels

🛠️ The `statsmodels` OLS API

Two equivalent interfaces

Formula interface (recommended for readability):

```
import statsmodels.formula.api as smf
model = smf.ols('response ~ predictor', data=df).fit()
```

Array interface (more control, used in MLR):

```
import statsmodels.api as sm
X = sm.add_constant(df['predictor'])  # add intercept column
model = sm.OLS(df['response'], X).fit()
```

We will use the formula interface

- The formula `'body_mass_g ~ flipper_length_mm'` directly maps to:

$$Y_i = \beta_0 + \beta_1 \cdot \text{flipper\_length}_i + \varepsilon_i.$$

- `statsmodels` automatically adds the intercept β_0.

Fitting the Model

One line of code!

```python
# Step 1: Specify and fit the model using the formula interface
model = smf.ols('body_mass_g ~ flipper_length_mm', data=penguins).fit()

# Step 2: Print the full regression summary
print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:            body_mass_g   R-squared:                       0.759
Model:                            OLS   Adj. R-squared:                  0.758
Method:                 Least Squares   F-statistic:                     1071.
Date:                Sat, 25 Apr 2026   Prob (F-statistic):          4.37e-107
Time:                        19:14:29   Log-Likelihood:                -2528.4
No. Observations:                 342   AIC:                             5061.
Df Residuals:                     340   BIC:                             5069.
Df Model:                           1
Covariance Type:            nonrobust
=====================================================================================
                        coef    std err          t      P>|t|      [0.025      0.975]
-------------------------------------------------------------------------------------
Intercept         -5780.8314    305.815    -18.903      0.000   -6382.358   -5179.305
flipper_length_mm    49.6856      1.518     32.722      0.000      46.699      52.672
==============================================================================
Omnibus:                        5.634   Durbin-Watson:                   2.176
Prob(Omnibus):                  0.060   Jarque-Bera (JB):                5.585
Skew:                           0.313   Prob(JB):                       0.0613
Kurtosis:                       3.019   Cond. No.                     2.89e+03
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 2.89e+03. This might indicate that there are
```

- Let's break down the information in the summary output.

Reading the Summary: Top Block

Model-level information

The **top block** reports global information about the fit:

Field	Meaning
`Dep. Variable`	The response variable Y
`No. Observations`	Sample size n
`R-squared`	Proportion of variance in Y explained by the model (R^2)
`Adj. R-squared`	R^2 adjusted for the number of predictors (more on this in Lecture 11)
`F-statistic`	Test of $H_0 : \beta_1 = 0$ (all slopes are zero) — equivalent to the t-test in SLR
`Prob (F-statistic)`	p-value for the F-test
`AIC / BIC`	Information criteria for model comparison (lower is better)
`Log-Likelihood`	Value of the log-likelihood at the MLE

> In SLR, `Prob (F-statistic)` and the p-value for $\hat{\beta}_1$ are **identical**. Both test whether $\beta_1 = 0$.

Reading the Summary: Coefficients Table

The most important section

```python
# Extract the coefficients table as a DataFrame
coef_table = model.summary2().tables[1]
print(coef_table.round(4))

# Intercept β₀
b0 = model.params['Intercept']
print(f"\nIntercept  β̂₀ = {b0:.2f} g")

# Slope β₁
b1 = model.params['flipper_length_mm']
print(f"Slope β̂₁ = {b1:.4f} g per mm")

# 95% confidence intervals
ci = model.conf_int()
print(f"\n95% CI for β̂₁: ({ci.loc['flipper_length_mm', 0]:.3f}, {ci.loc['flipper_length_mm', 1]:.3f})")
```

```
                     Coef.  Std.Err.        t  P>|t|     [0.025     0.975]
Intercept         -5780.8314  305.8145 -18.9031    0.0 -6382.3580 -5179.3047
flipper_length_mm    49.6856    1.5184  32.7222    0.0    46.6989    52.6722

Intercept  β̂₀ = -5780.83 g
Slope β̂₁ = 49.6856 g per mm

95% CI for β̂₁: (46.699, 52.672)
```

Interpreting Each Column

Column	Symbol	Meaning
`coef`	$\hat{\beta}_j$	OLS estimate
`std err`	$\widehat{\mathrm{SE}}(\hat{\beta}_j)$	Standard error of the estimate
`t`	$T = \hat{\beta}_j / \widehat{\mathrm{SE}}$	Test statistic for $H_0 : \beta_j = 0$
`P>|t|`	p-value	Evidence against H_0 — **not** the probability H_0 is true
`[0.025, 0.975]`	95% CI	Range of plausible values for β_j

Interpreting our estimates

- $\hat{\beta}_0 \approx -5781$ g: expected body mass when flipper length is 0 mm
  - **Not meaningful** (no penguin has 0 mm flippers); the intercept simply anchors the line.
- $\hat{\beta}_1 \approx 49.7$ g/mm: for each additional millimetre of flipper length.
  - Body mass is predicted to **increase** by ~49.7 g/mm on average.
- Both p-values are effectively **zero**.
  - **Strong evidence** that flipper length linearly predicts body mass.

Reading the Summary: Diagnostic Statistics

The bottom block

Field	What it tests
`Omnibus` / `Prob(Omnibus)`	Normality of residuals (combined skewness + kurtosis test)
`Skew`	Skewness of the residuals (ideal: ≈ 0)
`Kurtosis`	Kurtosis of the residuals (ideal: ≈ 3)
`Jarque-Bera (JB)` / `Prob(JB)`	Another normality test for residuals
`Durbin-Watson`	Tests for serial correlation in residuals (ideal: ≈ 2)
`Cond. No.`	Condition number — flags potential multicollinearity (relevant for MLR)

> These statistics give an initial indication — we will inspect assumptions **visually** through diagnostic plots in the next section.

Reading the Summary: Model Fit Statistics

R^2, F-test, and residual variance

```python
# Fitted values and residuals
y_hat = model.fittedvalues
resid = model.resid

# Model fit statistics
print(f"R² = {model.rsquared:.4f}")
print(f"Adj. R² = {model.rsquared_adj:.4f}")

# F-test
print(f"\nF-statistic = {model.fvalue:.2f}")
print(f"Prob(F) = {model.f_pvalue:.2e}")

# Residual std deviation (σ̂)
print(f"\nσ̂ (RMSE) = {np.sqrt(model.mse_resid):.2f} g")
print(f"n = {int(model.nobs)}, df_resid = {int(model.df_resid)}")
```

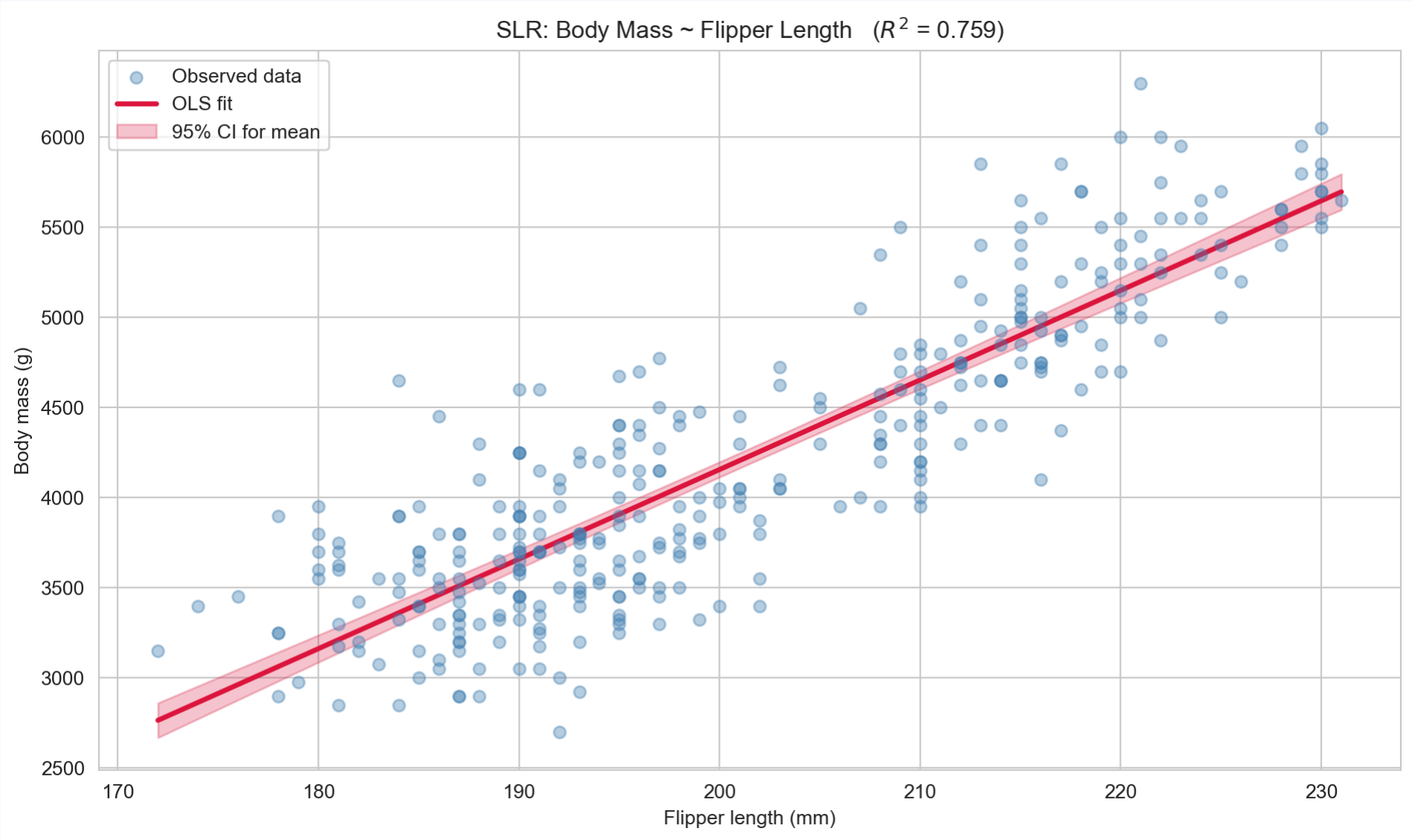
```
R² = 0.7590
Adj. R² = 0.7583

F-statistic = 1070.74
Prob(F) = 4.37e-107

σ̂ (RMSE) = 394.28 g
n = 342, df_resid = 340
```

Visualising the Fit

📈 Plotting the Regression Line

```
# Build a prediction grid
x_grid = pd.DataFrame({
    'flipper_length_mm': np.linspace(
        penguins['flipper_length_mm'].min(),
        penguins['flipper_length_mm'].max(), 200)
})

# Get predictions with confidence interval
pred_ci = model.get_prediction(x_grid).summary_frame(alpha=0.05)

fig, ax = plt.subplots(figsize=(10, 6))
ax.scatter(penguins['flipper_length_mm'], penguins['body_mass_g'],
           alpha=0.4, color='steelblue', s=35, label='Observed data', zorder=3)
ax.plot(x_grid['flipper_length_mm'], pred_ci['mean'],
        color='crimson', lw=2.5, label='OLS fit')
ax.fill_between(x_grid['flipper_length_mm'],
                pred_ci['mean_ci_lower'], pred_ci['mean_ci_upper'],
                alpha=0.25, color='crimson', label='95% CI for mean')
ax.set_xlabel('Flipper length (mm)'); ax.set_ylabel('Body mass (g)')
ax.set_title(f'SLR: Body Mass ~ Flipper Length   ($R^2$ = {model.rsquared:.3f})')
ax.legend(); plt.tight_layout(); plt.show()
```

📈 Plotting the Regression Line

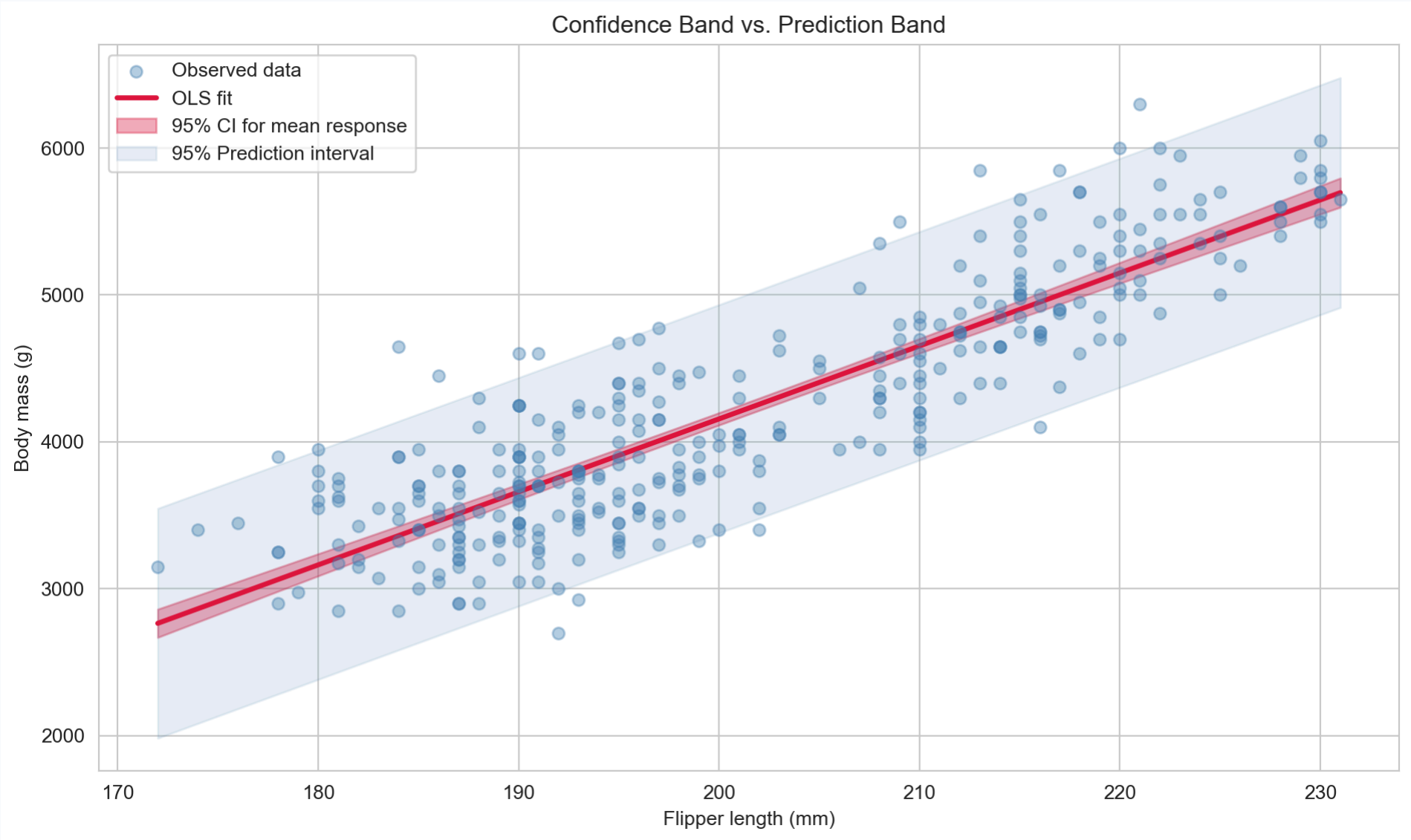


OLS regression line fitted to the penguins data. The shaded region is the 95% confidence band for the mean response.

Adding the Prediction Interval

```
pred_pi = model.get_prediction(x_grid).summary_frame(alpha=0.05)

fig, ax = plt.subplots(figsize=(10, 6))
ax.scatter(penguins['flipper_length_mm'], penguins['body_mass_g'],
           alpha=0.4, color='steelblue', s=35, label='Observed data', zorder=3)
ax.plot(x_grid['flipper_length_mm'], pred_pi['mean'],
        color='crimson', lw=2.5, label='OLS fit')
ax.fill_between(x_grid['flipper_length_mm'],
                pred_pi['mean_ci_lower'], pred_pi['mean_ci_upper'],
                alpha=0.35, color='crimson', label='95% CI for mean response')
ax.fill_between(x_grid['flipper_length_mm'],
                pred_pi['obs_ci_lower'], pred_pi['obs_ci_upper'],
                alpha=0.12, color='steelblue', label='95% Prediction interval')
ax.set_xlabel('Flipper length (mm)'); ax.set_ylabel('Body mass (g)')
ax.set_title('Confidence Band vs. Prediction Band')
ax.legend(); plt.tight_layout(); plt.show()
```

Adding the Prediction Interval



The prediction interval (light fill) is wider than the confidence band because it accounts for individual observation error.

Making a Specific Prediction

Using the model for a new observation

```python
# A new penguin with flipper length 195 mm
new_penguin = pd.DataFrame({'flipper_length_mm': [195]})

# Point prediction
y_hat_new = model.predict(new_penguin)
print(f"Predicted mass: {y_hat_new.values[0]:.1f} g")

# Full prediction with intervals
pred_new = (
  model.get_prediction(new_penguin)
  .summary_frame(alpha=0.05)
  )
print(f"\n95% CI for mean:  ({pred_new['mean_ci_lower'].values[0]:.1f},"
      f" {pred_new['mean_ci_upper'].values[0]:.1f}) g")
print(f"95% PI for obs:   ({pred_new['obs_ci_lower'].values[0]:.1f},"
      f" {pred_new['obs_ci_upper'].values[0]:.1f}) g")
```

```
Predicted mass: 3907.9 g

95% CI for mean:  (3862.3, 3953.4) g
95% PI for obs:   (3131.0, 4684.7) g
```

- The **CI** tells us: we are 95% confident the *average* penguin with 195 mm flippers has body mass in that range.
- The **PI** tells us: we expect a *single* new penguin with 195 mm flippers to land in the wider interval 95% of the time.

Residual Diagnostics

🔬 Why Check Assumptions?

- We derived the nice properties of OLS (unbiasedness, BLUE, exact t-inference) assuming GM1–GM4.
- If the assumptions are violated, our estimates may be **biased**, our standard errors **wrong**, and our p-values **misleading**.
- We can never verify assumptions perfectly, but **residual plots** give us the best available evidence.

The four diagnostic plots we will produce

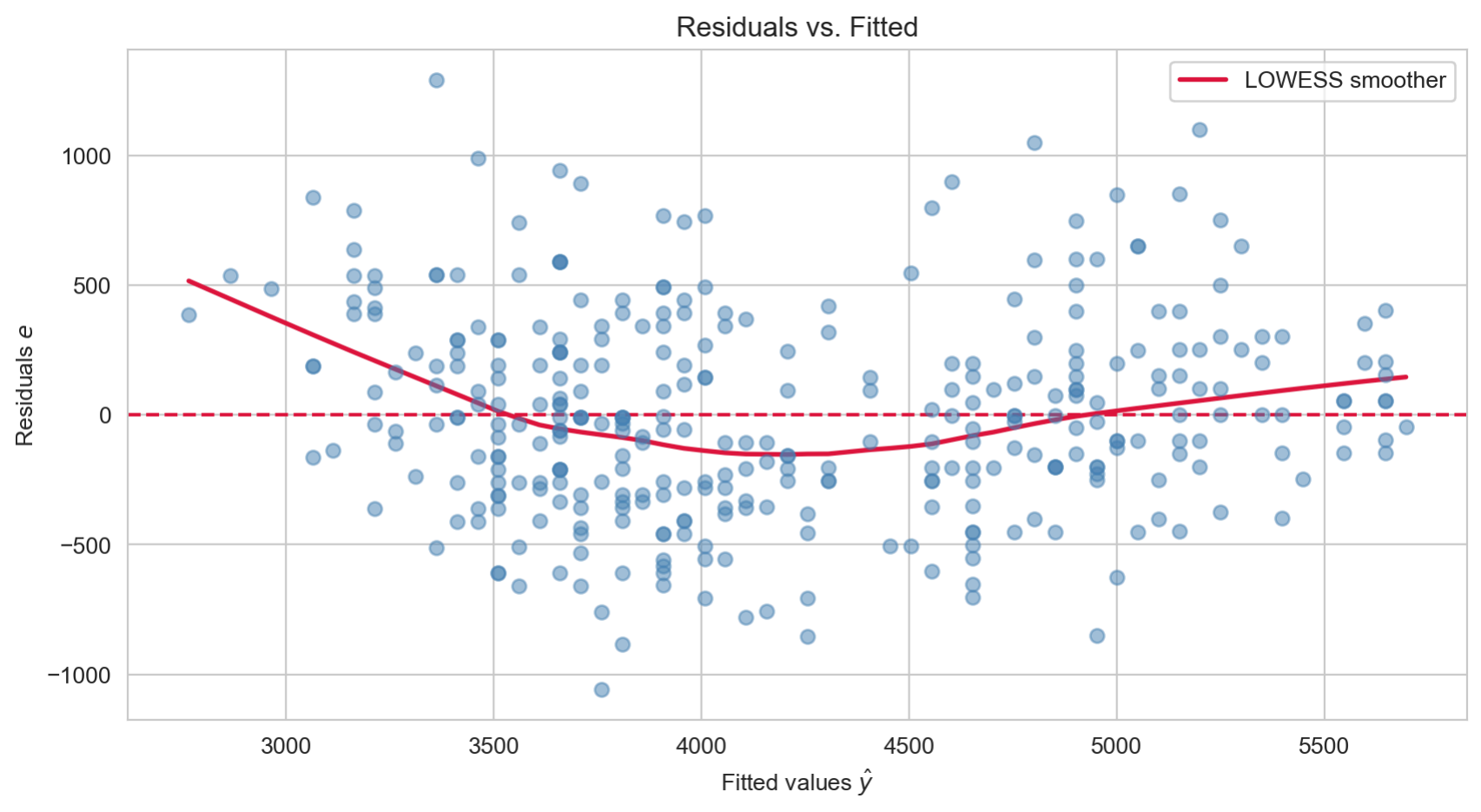
Plot	Assumption checked
Residuals vs. Fitted	Linearity (GM1) and homoscedasticity (GM3)
Normal Q-Q	Normality of errors (GM4)
Scale-Location	Homoscedasticity (GM3) — a different view
Cook's Distance	Influential observations

Residuals vs. Fitted

Checking linearity and homoscedasticity

```
# Extract fitted values and residuals
fitted   = model.fittedvalues
residual = model.resid

fig, ax = plt.subplots(figsize=(9, 5))
ax.scatter(fitted, residual, alpha=0.5, color='steelblue', s=35, zorder=3)
ax.axhline(0, color='crimson', lw=1.5, linestyle='--')
lowess = sm.nonparametric.lowess(residual, fitted, frac=0.5)
ax.plot(lowess[:, 0], lowess[:, 1], color='crimson', lw=2, label='LOWESS smoother')
ax.set_xlabel('Fitted values $\\hat{y}$'); ax.set_ylabel('Residuals $e$')
ax.set_title('Residuals vs. Fitted'); ax.legend()
plt.tight_layout(); plt.show()
```

Residuals vs. Fitted



Residuals vs. fitted values. We want random scatter around the horizontal zero line with no systematic pattern.

Residuals vs. Fitted — What to Look For

A well-behaved plot shows:

- Points scattered **randomly** around the horizontal line at zero.
- **No curvature** — the red smoother should be approximately flat.
- **Constant vertical spread** — no fanning out or narrowing.

Warning signs

Pattern	Likely violation	Action
Systematic curve	Linearity (GM1) — the true relationship is nonlinear	Add polynomial term
Fan / funnel shape	Homoscedasticity (GM3)	Log-transform Y; use WLS
Striped bands	Possible discrete/omitted variable	Add grouping variable

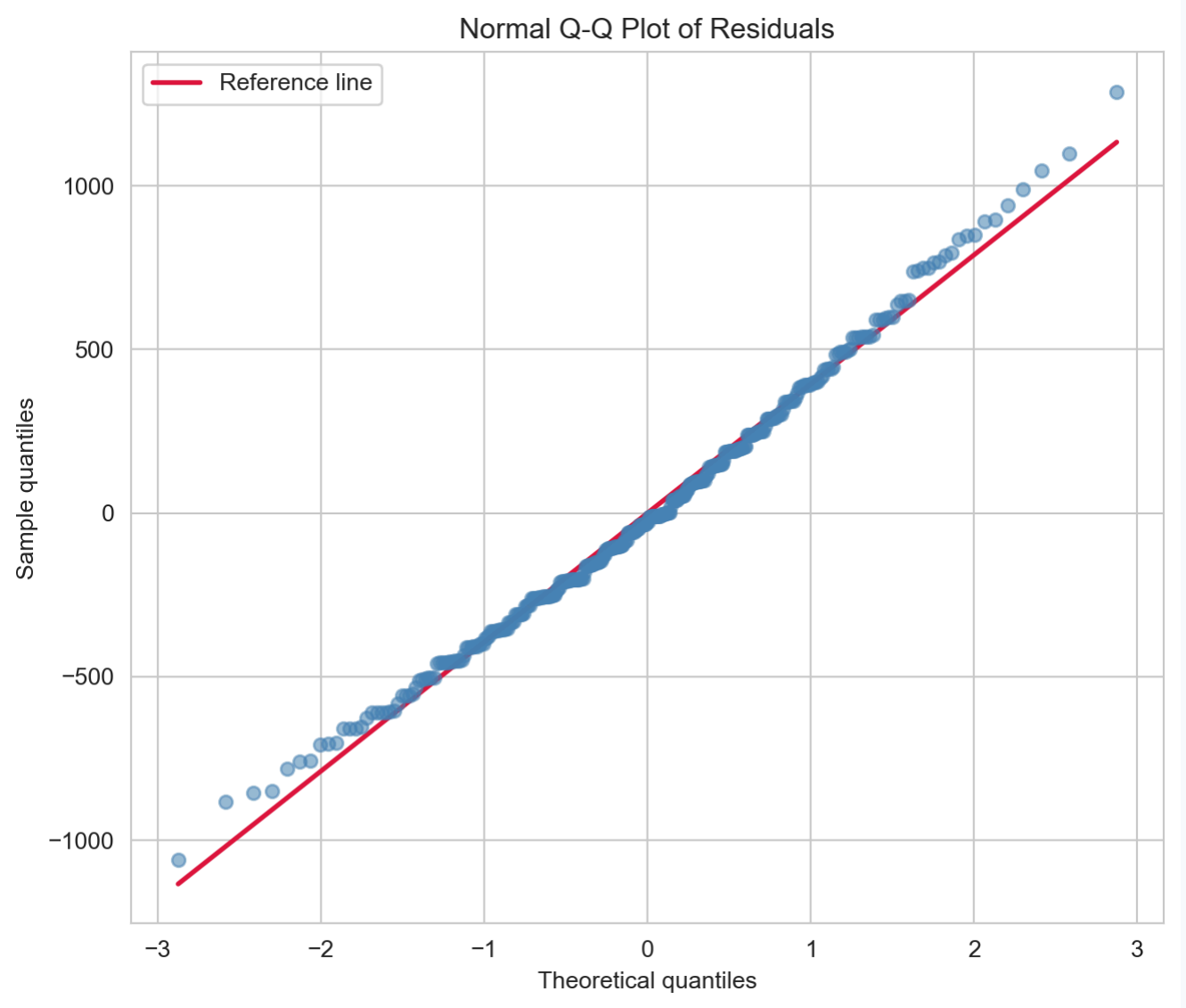
> Our penguins plot shows mild **vertical banding** — a hint that an omitted variable (species) is creating structure in the residuals. This will motivate adding species as a predictor in Lecture 11.

Normal Q-Q Plot

Checking normality of errors (GM4)

```
from scipy.stats import probplot

qq_result = probplot(residual, dist='norm')

fig, ax = plt.subplots(figsize=(7, 6))
ax.scatter(qq_result[0][0], qq_result[0][1],
           alpha=0.55, color='steelblue', s=30, zorder=3)
ax.plot(qq_result[0][0],
        qq_result[1][1] + qq_result[1][0] * qq_result[0][0],
        color='crimson', lw=2, label='Reference line')
ax.set_xlabel('Theoretical quantiles'); ax.set_ylabel('Sample quantiles')
ax.set_title('Normal Q-Q Plot of Residuals'); ax.legend()
plt.tight_layout(); plt.show()
```

Normal Q-Q Plot



Normal Q-Q plot of residuals. Points should lie close to the diagonal if errors are normally distributed.

Normal Q-Q Plot — Interpretation

Well-behaved residuals lie close to the diagonal

Pattern	Interpretation
Points on the line	Residuals are approximately normal ✓
Heavy tails (S-curve)	Leptokurtic — more extreme values than normal
Light tails	Platykurtic — fewer extreme values than normal
Systematic curve	Skewed residuals — consider transformation

Why normality matters — and when it doesn't

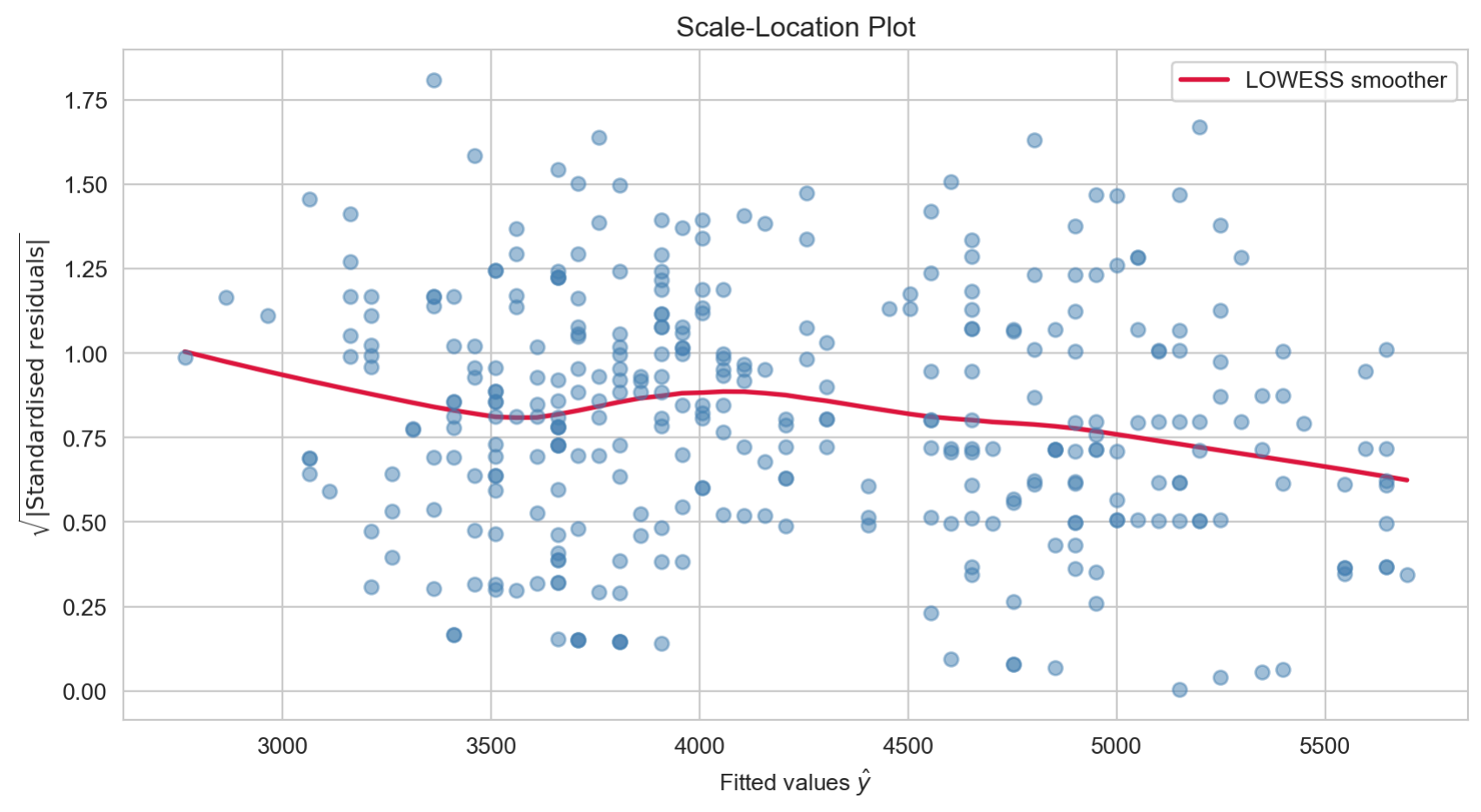
- Normality is required for **exact** t-inference in small samples.
- For $n \geq 30$, the **CLT** means the t-tests are approximately valid regardless.
- A slight departure from normality in a sample of $n = 333$ is not a serious concern.
- Extreme outliers or strong skew in the residuals are worth investigating.

Scale-Location Plot

A second view of homoscedasticity

```
# Standardised residuals
resid_std = residual / np.sqrt(np.sum(residual**2) / (len(residual) - 2))
sqrt_abs  = np.sqrt(np.abs(resid_std))

fig, ax = plt.subplots(figsize=(9, 5))
ax.scatter(fitted, sqrt_abs, alpha=0.5, color='steelblue', s=35, zorder=3)
lowess = sm.nonparametric.lowess(sqrt_abs, fitted, frac=0.5)
ax.plot(lowess[:, 0], lowess[:, 1], color='crimson', lw=2, label='LOWESS smoother')
ax.set_xlabel('Fitted values $\\hat{y}$')
ax.set_ylabel('$\\sqrt{|\\text{Standardised residuals}|}$')
ax.set_title('Scale-Location Plot'); ax.legend()
plt.tight_layout(); plt.show()
```

Scale-Location Plot



Scale-location plot: square root of |standardised residuals| vs. fitted values. A horizontal red smoother indicates constant variance.

Cook's Distance — Influential Observations

Which points are driving the fit?

Cook's Distance

Cook's distance D_i measures how much the fitted values change if observation i is removed. A large D_i indicates an **influential** point — one that substantially affects the estimated coefficients.

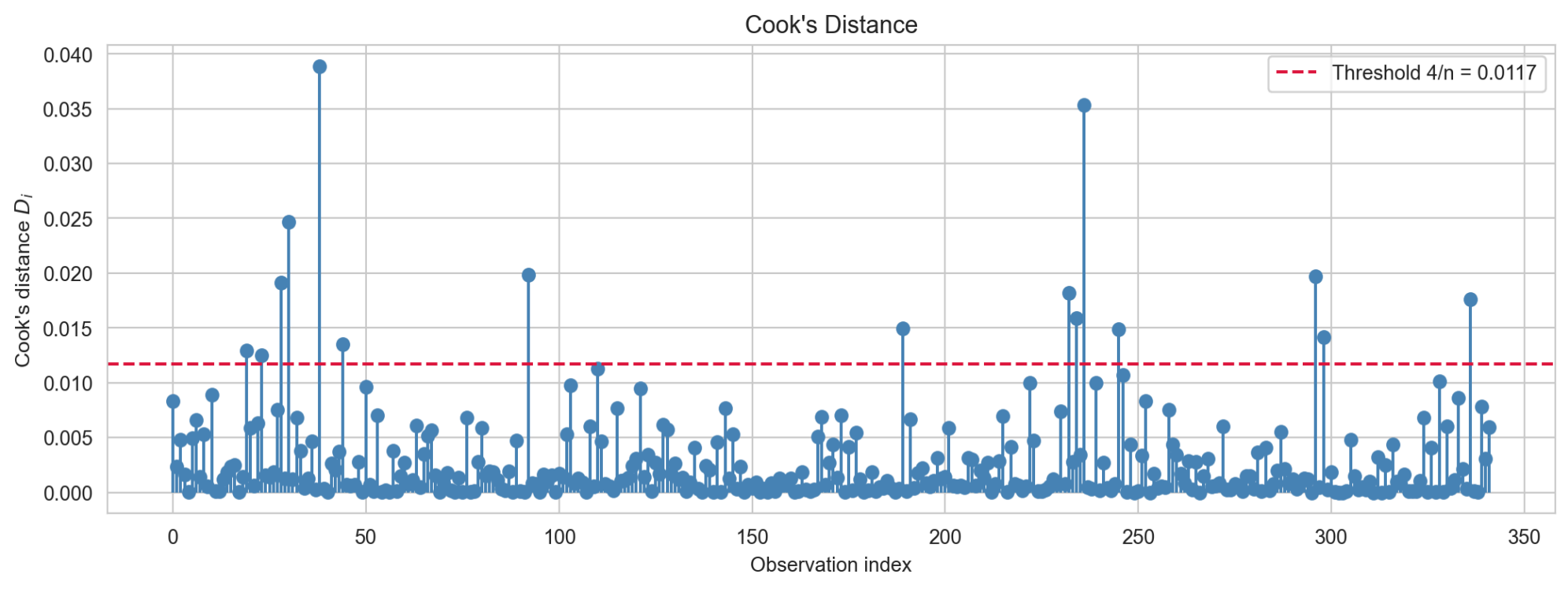
- **Leverage**: how unusual is x_i relative to the other predictor values?
- **Outlier**: how large is the residual e_i?
- **Influential** = high leverage AND large residual.
- A common threshold: $D_i > 4/n$ warrants investigation.

Cook's Distance in Python

```
# Compute influence measures via statsmodels
influence = model.get_influence()
cooks_d   = influence.cooks_distance[0]
threshold = 4 / len(penguins)

fig, ax = plt.subplots(figsize=(11, 4))
ax.stem(range(len(cooks_d)), cooks_d,
        linefmt='steelblue', markerfmt='o', basefmt=' ')
ax.axhline(threshold, color='crimson', linestyle='--',
           label=f'Threshold 4/n = {threshold:.4f}')

# Flag influential points
flagged = np.where(cooks_d > threshold)[0]
print(f"Observations above threshold: {len(flagged)}")
ax.set_xlabel('Observation index'); ax.set_ylabel("Cook's distance $D_i$")
ax.set_title("Cook's Distance"); ax.legend()
plt.tight_layout(); plt.show()
```

Cook's Distance in Python

```
Observations above threshold: 15
```

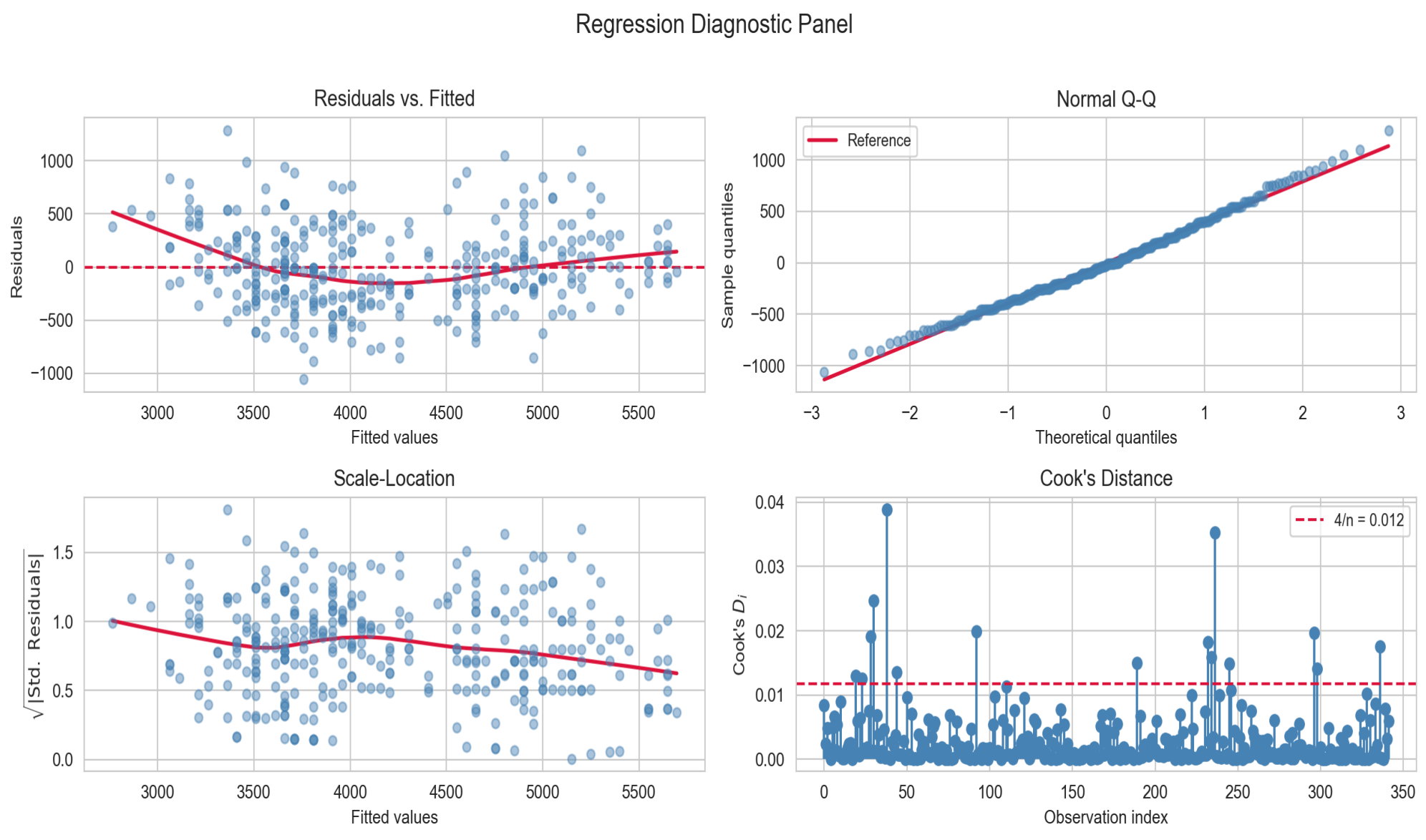


Cook's distance for each observation. Points above the threshold (red dashed line) have disproportionate influence on the fit.

The Full Diagnostic Panel

```
fig, axes = plt.subplots(2, 2, figsize=(13, 6))

# 1. Residuals vs Fitted
axes[0,0].scatter(fitted, residual, alpha=0.45, color='steelblue', s=25, zorder=3)
axes[0,0].axhline(0, color='crimson', lw=1.5, linestyle='--')
lw1 = sm.nonparametric.lowess(residual, fitted, frac=0.5)
axes[0,0].plot(lw1[:,0], lw1[:,1], 'crimson', lw=2)
axes[0,0].set_xlabel('Fitted values'); axes[0,0].set_ylabel('Residuals')
axes[0,0].set_title('Residuals vs. Fitted')

# 2. Normal Q-Q
qq = probplot(residual, dist='norm')
axes[0,1].scatter(qq[0][0], qq[0][1], alpha=0.5, color='steelblue', s=25, zorder=3)
axes[0,1].plot(qq[0][0], qq[1][1] + qq[1][0]*qq[0][0], 'crimson', lw=2, label='Reference')
axes[0,1].set_xlabel('Theoretical quantiles'); axes[0,1].set_ylabel('Sample quantiles')
axes[0,1].set_title('Normal Q-Q'); axes[0,1].legend(fontsize=9)

# 3. Scale-Location
axes[1,0].scatter(fitted, sqrt_abs, alpha=0.45, color='steelblue', s=25, zorder=3)
lw2 = sm.nonparametric.lowess(sqrt_abs, fitted, frac=0.5)
axes[1,0].plot(lw2[:,0], lw2[:,1], 'crimson', lw=2)
axes[1,0].set_xlabel('Fitted values')
axes[1,0].set_ylabel('$\\sqrt{|\\mathrm{Std.\\ Residuals}|}$')
axes[1,0].set_title('Scale-Location')

# 4. Cook's Distance
axes[1,1].stem(range(len(cooks_d)), cooks_d,
               linefmt='steelblue', markerfmt='o', basefmt=' ')
axes[1,1].axhline(threshold, color='crimson', linestyle='--', label=f'4/n = {threshold:.3f}')
```

The Full Diagnostic Panel



A 2×2 diagnostic panel: residuals vs. fitted (top left), Q-Q plot (top right), scale-location (bottom left), Cook's distance (bottom right).

Anscombe's Quartet

🎭 Why Plots Are Not Optional

Four datasets with identical statistics

In 1973, statistician Francis Anscombe constructed four datasets that share **the same summary statistics** but look completely different visually (Anscombe 1973):

```
anscombe = sns.load_dataset('anscombe')
for ds, grp in anscombe.groupby('dataset'):
    r = grp['x'].corr(grp['y'])
    print(f"Dataset {ds}: x̄={grp['x'].mean():.2f}, ȳ={grp['y'].mean():.2f}, "
          f"r={r:.3f}, R²={r**2:.3f}")
```

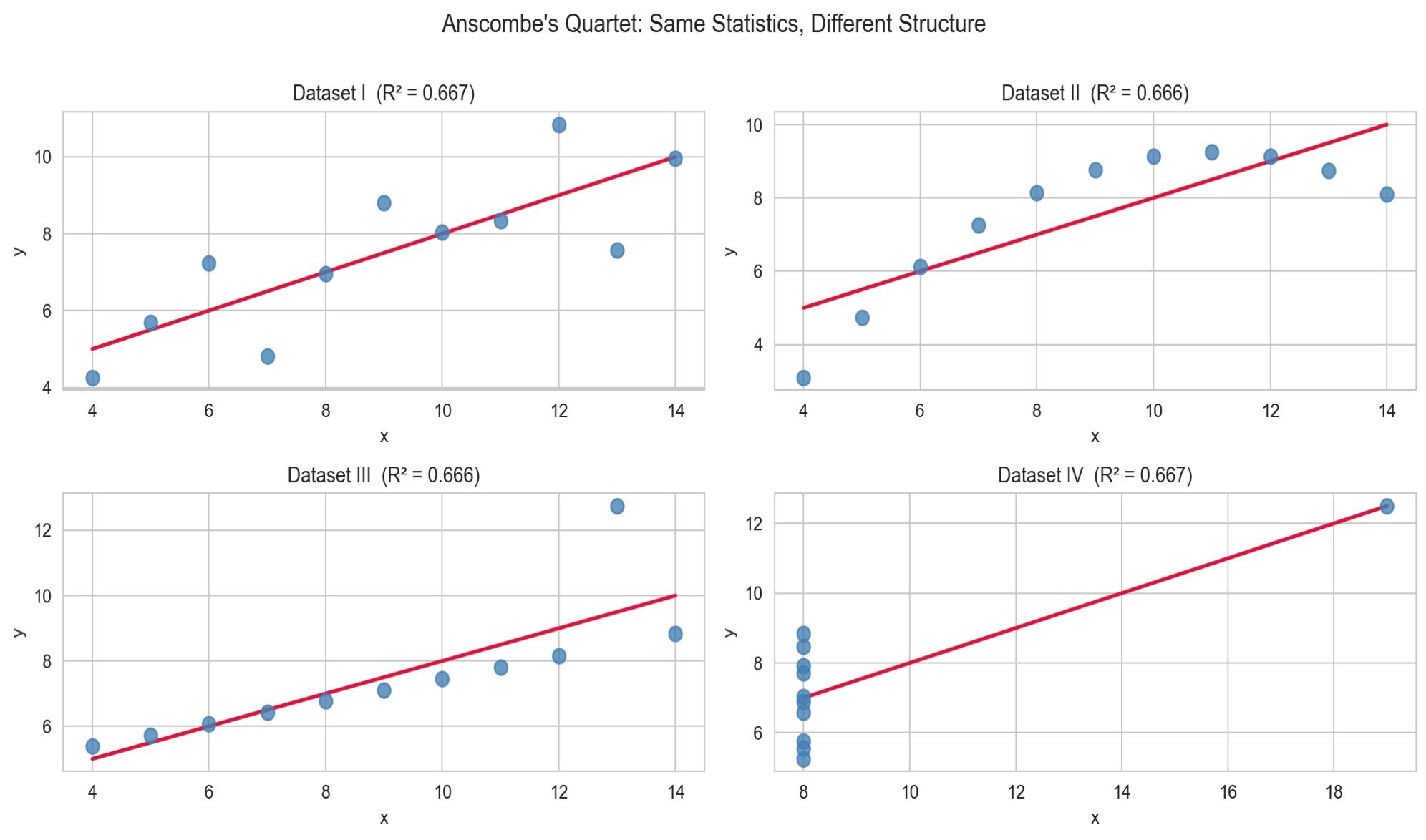
```
Dataset I: x̄=9.00, ȳ=7.50, r=0.816, R²=0.667
Dataset II: x̄=9.00, ȳ=7.50, r=0.816, R²=0.666
Dataset III: x̄=9.00, ȳ=7.50, r=0.816, R²=0.666
Dataset IV: x̄=9.00, ȳ=7.50, r=0.817, R²=0.667
```

- Same means, same R^2, same regression line coefficients — yet the data are fundamentally different.

Anscombe's Quartet — Visualised

```
fig, axes = plt.subplots(2, 2, figsize=(12, 6), sharex=False, sharey=False)

for ax, (ds, grp) in zip(axes.flat, anscombe.groupby('dataset')):
    model_a = smf.ols('y ~ x', data=grp).fit()
    x_a     = np.linspace(grp['x'].min(), grp['x'].max(), 100)
    y_a     = model_a.params['Intercept'] + model_a.params['x'] * x_a

    ax.scatter(grp['x'], grp['y'], color='steelblue', s=60, zorder=3, alpha=0.8)
    ax.plot(x_a, y_a, color='crimson', lw=2)
    ax.set_title(f"Dataset {ds}  (R² = {model_a.rsquared:.3f})", fontsize=11)
    ax.set_xlabel('x'); ax.set_ylabel('y')

plt.suptitle("Anscombe's Quartet: Same Statistics, Different Structure", fontsize=13, y=1.01)
plt.tight_layout(); plt.show()
```

Anscombe's Quartet — Visualised



Anscombe's quartet: four datasets with identical regression statistics but very different structure. Only dataset I is appropriate for SLR.

Lessons from Anscombe's Quartet

Dataset	What is actually happening	What to do
I	Linear relationship + random noise — SLR is appropriate	Proceed normally
II	Curved / nonlinear relationship	Add polynomial term; use different model
III	Linear relationship with one influential outlier	Investigate the outlier; consider robust regression
IV	One extreme leverage point drives everything	Investigate the point; do not report the regression uncritically

The key message

> **Always plot your data and your residuals.** Summary statistics alone — including R^2 and p-values — can be identical for datasets with completely different structures.

Putting It All Together

The Complete SLR Workflow

Step	Code	Purpose
1. Load & inspect	`sns.load_dataset(); .describe(); .head()`	Understand the data
2. Visualise	`plt.scatter(); sns.pairplot()`	Check linearity, outliers, subgroups
3. Fit	`smf.ols('y ~ x', data=df).fit()`	Estimate $\hat{\beta}_0, \hat{\beta}_1$
4. Summarise	`model.summary()`	Read coefficients, p-values, R^2
5. Plot fit	`model.get_prediction().summary_frame()`	Visualise line + CI/PI bands
6. Diagnostics	Residuals vs. fitted, Q-Q, scale-location, Cook's	Check assumptions
7. Report	Coefficient, SE, CI, p-value, R^2	Communicate results honestly

A Note on What SLR Cannot Tell Us

What we found

- Flipper length is a strong predictor of body mass ($R^2 \approx 0.76, p < 0.001$).
- The estimated slope is ~49.7 g per mm.

What we should not conclude

- **Causation**: a longer flipper does not *cause* greater mass — both may be driven by species, age, diet, or other factors.
- **Completeness**: the residual diagnostic hints that **species** is an omitted variable creating structure in the residuals.
- **Extrapolation**: the model is only reliable within the observed range of flipper lengths (~170–235 mm).

> These limitations motivate **multiple linear regression** — adding species and other predictors to better model the joint structure. That is the topic of **Lecture 11**.

Conclusion

✅ What we covered

- **Loading and exploring** a real dataset in preparation for regression.
- **Fitting SLR** with `statsmodels` using the formula interface.
- **Reading the regression summary**: model info, coefficients table, and diagnostic statistics.
- **Visualising the fit**: the regression line, confidence bands, and prediction intervals.
- **Residual diagnostics**: residuals vs. fitted, Q-Q plot, scale-location, and Cook's distance.
- **Anscombe's quartet**: a compelling demonstration of why plots are essential.

📅 What's next?

- **Lecture 11 — Multiple Linear Regression**: adding more predictors, the matrix formulation, partial slopes, adjusted R^2, and model comparison.
- We will extend the penguins analysis by incorporating species and other covariates — and see how the diagnostics improve.

References

Anscombe, F. J. 1973. “Graphs in Statistical Analysis.” *The American Statistician* 27 (1): 17–21. https://doi.org/10.1080/00031305.1973.10478966.

Gorman, Kristen B., Tony D. Williams, and William R. Fraser. 2014. “Ecological Sexual Dimorphism and Environmental Variability Within a Community of Antarctic Penguins.” *PLOS ONE* 9 (3): e90081. https://doi.org/10.1371/journal.pone.0090081.